Modulo 2

Angelica Collazo

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

##Primeros comandos de R

R es un lenguaje muy similar a MatLab y permite el trabajo con matrices.

```
A=38
A=40
B=60
C=B-A
C
```

```
## [1] 20
```

Uso de datasets o base de datos internos de R

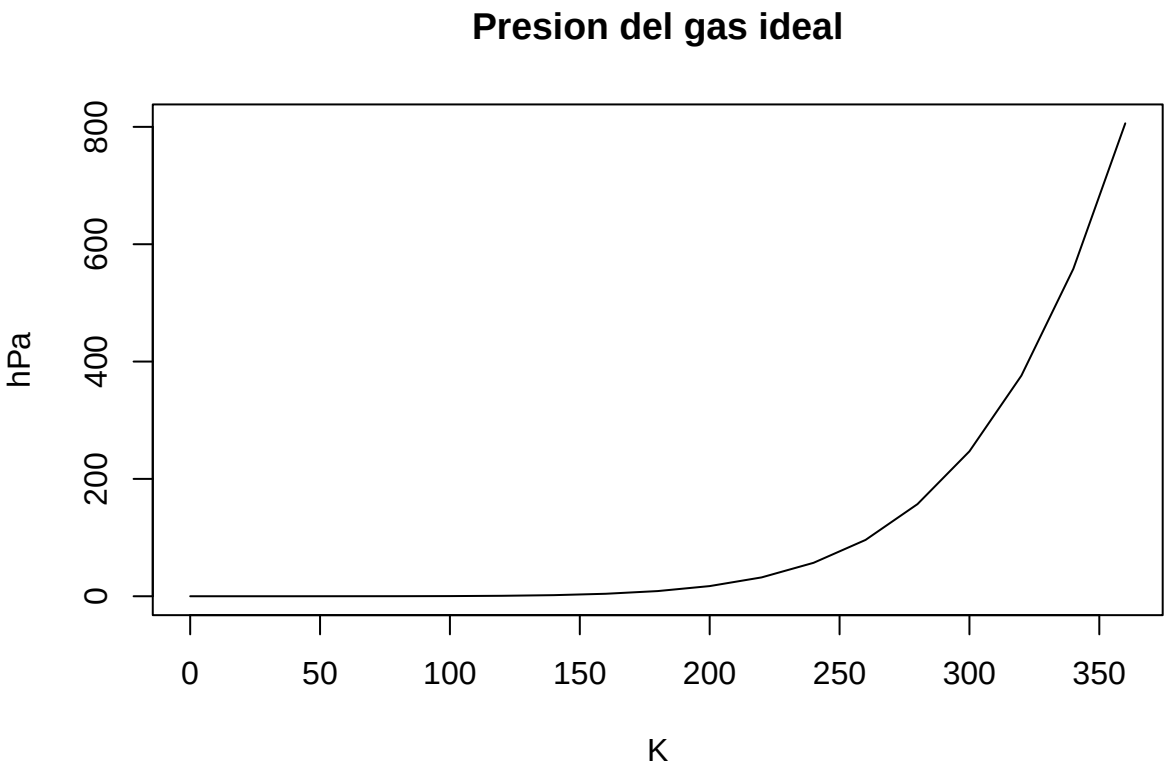
Usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos $, luego indicando la informacion especifica que deseamos.

```
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

Comando Plot

Es un comando que permite graficar cualquier fuente de datos que le asignemos, teniendo también la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar el comando podemos escribir **?plot** en la consola, lo cual abrira una ventana que explicara en detalle el comando.



Para que en grafico me aparezca la funcion con una linea, o puntos o de cualquier forma que yo quiera,escribo **type=letra** para saber que significa cada letra puedo escribir en la consola ?plot.

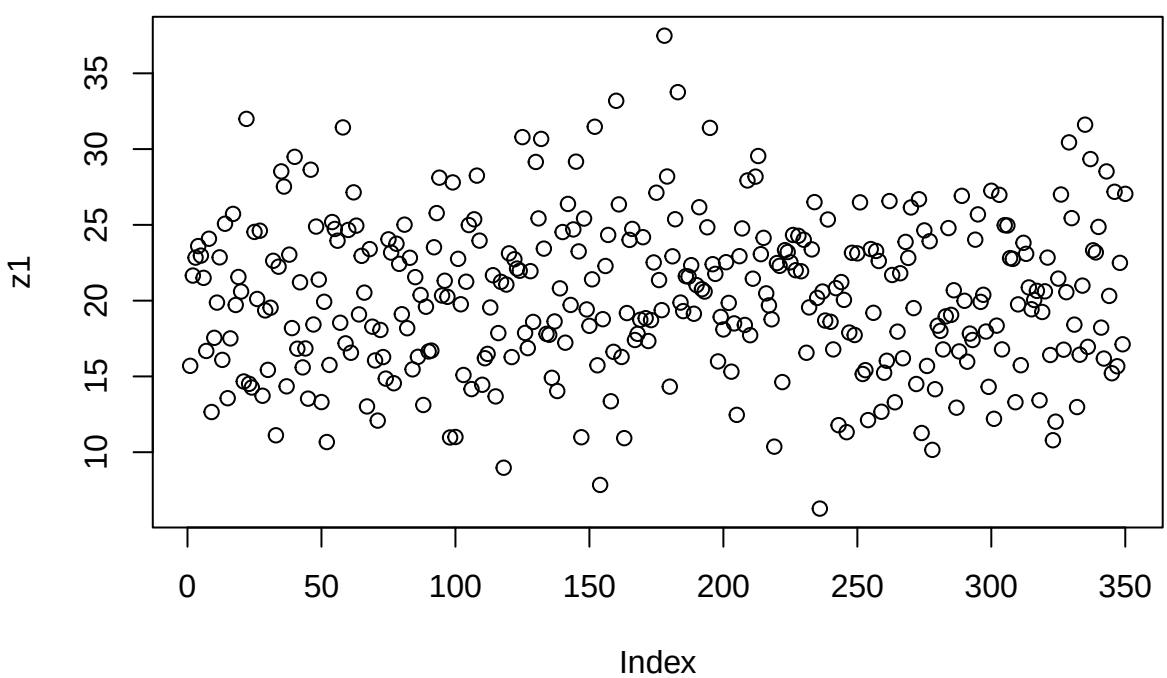
Funciones Estadisticas

```
z1<-rnorm(350,20,5)
z1
```

```
##   [1] 15.698711 21.656200 22.839562 23.599023 22.971851 21.503354 16.680789
##   [8] 24.084982 12.652353 17.547176 19.871231 22.857901 16.091760 25.073550
##  [15] 13.563891 17.507777 25.723741 19.714310 21.561965 20.601408 14.666168
##  [22] 31.993134 14.512267 14.269129 24.534940 20.110273 24.615558 13.725260
##  [29] 19.356470 15.431228 19.532563 22.627909 11.116295 22.241936 28.528555
##  [36] 27.527813 14.346956 23.037282 18.188757 29.488614 16.828705 21.210579
##  [43] 15.595253 16.843875 13.534230 28.641137 18.426900 24.887606 21.390207
##  [50] 13.308021 19.925261 10.670217 15.756472 25.186225 24.730724 23.954703
##  [57] 18.540414 31.428846 17.182522 24.665794 16.561720 27.144663 24.949456
##  [64] 19.090920 22.957941 20.529192 13.019738 23.401629 18.281525 16.052202
##  [71] 12.084924 18.066586 16.277783 14.857174 24.045247 23.170254 14.550107
##  [78] 23.755051 22.428430 19.103764 25.014477 18.181599 22.828104 15.467480
##  [85] 21.551934 16.291865 20.375221 13.116761 19.597918 16.646643 16.703263
##  [92] 23.521463 25.776055 28.115000 20.337277 21.313211 20.252828 10.972528
##  [99] 27.800934 11.002316 22.758332 19.759584 15.095218 21.249163 24.988076
## [106] 14.165139 25.366740 28.250099 23.960109 14.450399 16.200549 16.486918
## [113] 19.549923 21.677471 13.681609 17.860804 21.238502  8.976689 21.061054
## [120] 23.121389 16.276725 22.731551 22.116273 21.974231 30.790243 17.870704
## [127] 16.868820 21.945451 18.596167 29.150710 25.426480 30.671335 23.433098
## [134] 17.808248 17.748119 14.907803 18.626785 14.037189 20.810908 24.527262
## [141] 17.226094 26.381226 19.715956 24.698854 29.172001 23.248167 10.987458
## [148] 25.417920 19.419907 18.338878 21.406885 31.473455 15.733739  7.841676
```

```
## [155] 18.777208 22.286921 24.334746 13.361593 16.622921 33.186595 26.352363
## [162] 16.282618 10.926534 19.181176 24.008389 24.732224 17.401629 17.811179
## [169] 18.737719 24.186931 18.843597 17.335084 18.716387 22.518698 27.122512
## [176] 21.358654 19.373177 37.481883 28.192098 14.325148 22.925808 25.370740
## [183] 33.756163 19.873397 19.310620 21.616657 21.586205 22.339347 19.134850
## [190] 21.023309 26.164693 20.754454 20.599582 24.840769 31.399925 22.404453
## [197] 21.765872 15.990056 18.924798 18.098734 22.536816 19.852355 15.313519
## [204] 18.498506 12.464120 22.934774 24.765833 18.395839 27.929738 17.721491
## [211] 21.449018 28.184007 29.543383 23.067087 24.143094 20.480264 19.685726
## [218] 18.768448 10.365491 22.480528 22.296539 14.625267 23.330529 23.207977
## [225] 22.531350 24.334051 22.006592 24.277185 21.940122 24.018056 16.560917
## [232] 19.553342 23.382321 26.500839 20.181706  6.279230 20.600794 18.687125
## [239] 25.355179 18.604907 16.782445 20.825104 11.783602 21.225653 20.050089
## [246] 11.321746 17.895607 23.158376 17.733990 23.119068 26.486158 15.160355
## [253] 15.414280 12.119123 23.404221 19.197126 23.279253 22.615890 12.668918
## [260] 15.245096 16.034057 26.569741 21.693451 13.294599 17.954554 21.808816
## [267] 16.199648 23.875929 22.822362 26.147429 19.508026 14.499926 26.690521
## [274] 11.267315 24.634345 15.692844 23.922459 10.154547 14.156732 18.348489
## [281] 18.003463 16.781654 18.964105 24.788567 19.061605 20.685366 12.938567
## [288] 16.648951 26.913752 19.991005 15.972856 17.826670 17.403333 24.023412
## [295] 25.690972 19.918651 20.386249 17.961821 14.311875 27.252982 12.203279
## [302] 18.351459 26.981449 16.783235 24.980515 24.958243 22.806213 22.754248
## [309] 13.298193 19.753272 15.739353 23.816683 23.089053 20.872097 19.430592
## [316] 20.045217 20.646495 13.426469 19.251654 20.614819 22.841838 16.413485
## [323] 10.790013 12.014242 21.454211 27.002197 16.765014 20.559693 30.443118
## [330] 25.443831 18.417147 12.978974 16.432637 20.992354 31.611521 16.952277
## [337] 29.340601 23.321620 23.182519 24.864730 18.220418 16.180143 28.526661
## [344] 20.313376 15.210496 27.184552 15.671699 22.492027 17.120858 27.045710
```

```r
plot(z1)
```



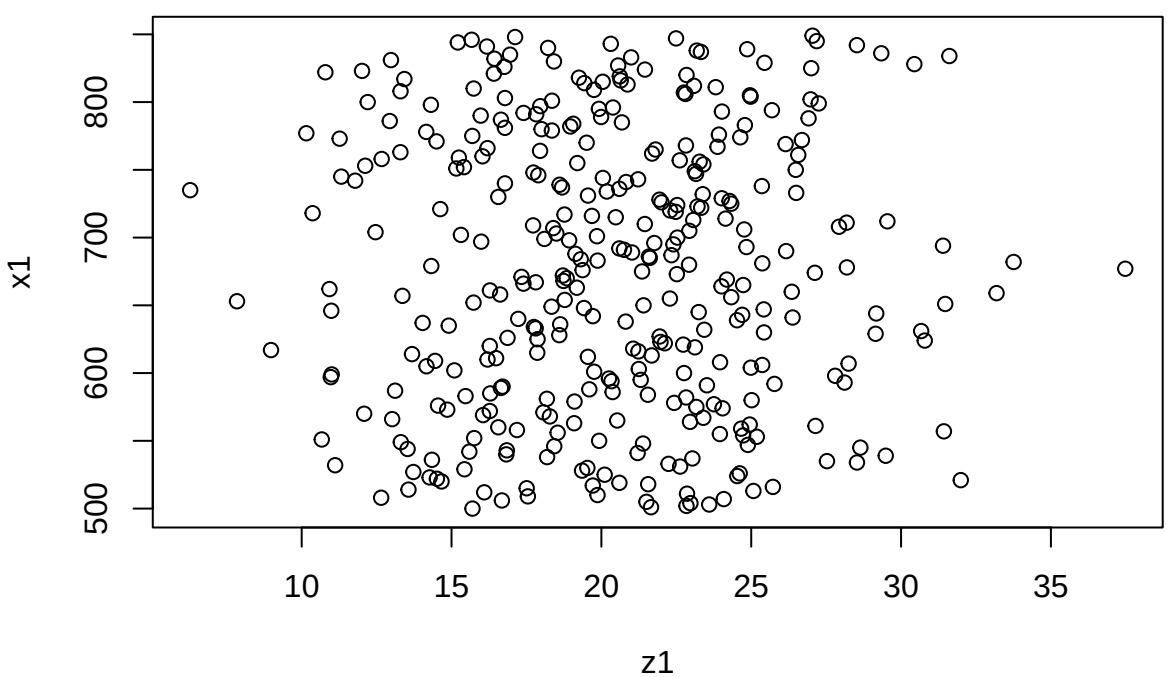

```r
w1<-length(z1)
w1
```

```
## [1] 350
```

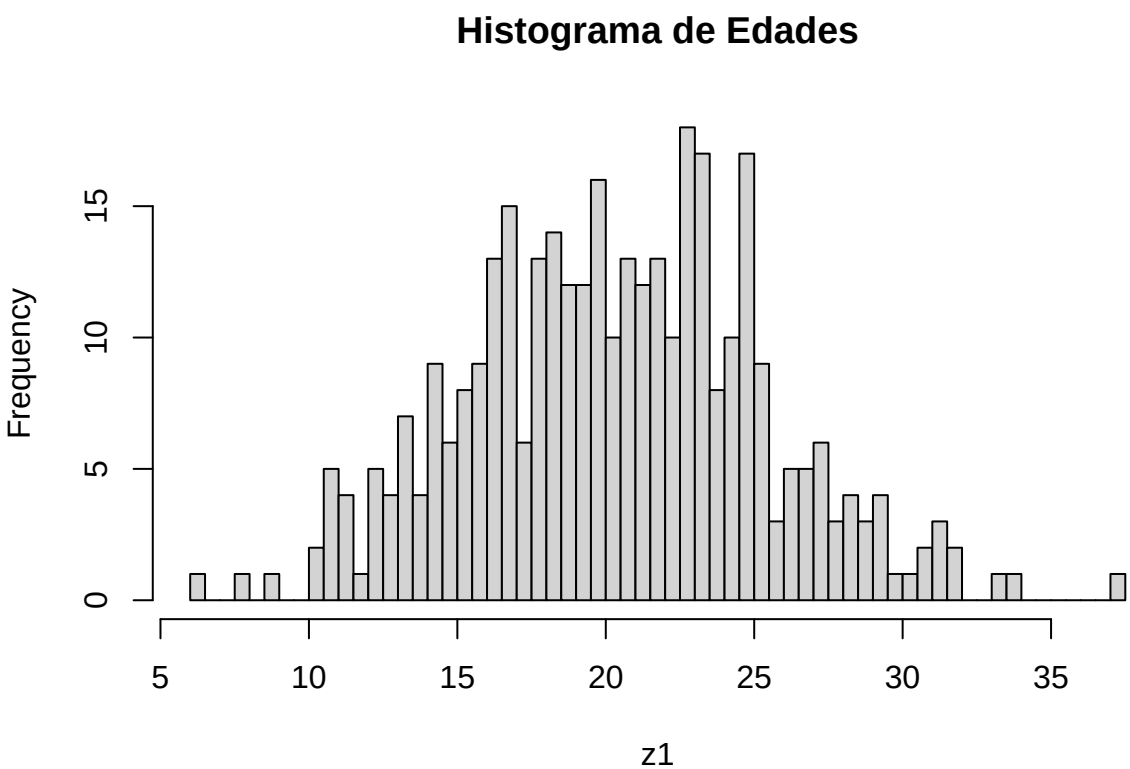
```r
x1<-(500:849)
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1,x1)
```



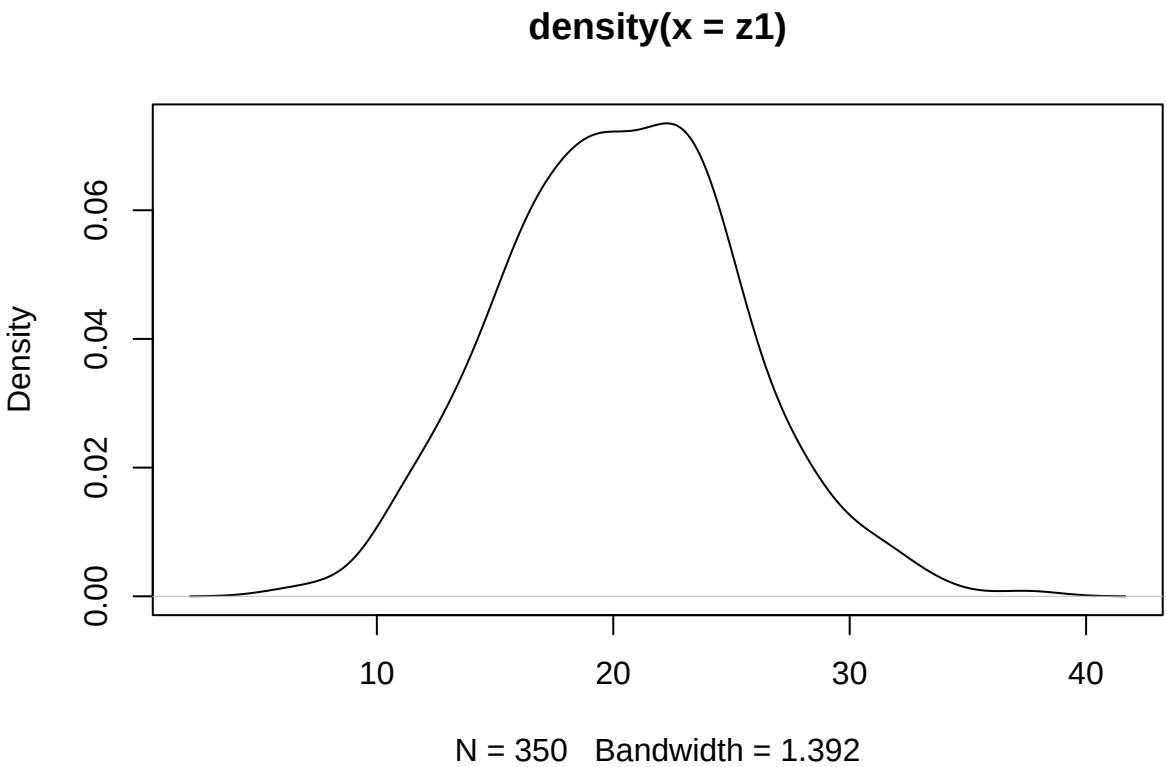

```r
hist(z1,main="Histograma de Edades",breaks=50)
```




```r
density(z1)
```

```
##
## Call:
##  density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.392
##
##        x                y
##  Min.   : 2.103   Min.   :9.150e-06
##  1st Qu.:11.992   1st Qu.:1.040e-03
##  Median :21.881   Median :1.236e-02
##  Mean   :21.881   Mean   :2.523e-02
##  3rd Qu.:31.769   3rd Qu.:5.025e-02
##  Max.   :41.658   Max.   :7.350e-02
```

```r
plot(density(z1))
```



El tamaño de muestra es de 350, valor promedio de 20 y un desvio estandar de 5.Si tengo alguna duda escribo en la consola **?rnorm**.

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Ejercicio 1 de algoritmo

Consigna: Las ultimas 3 cifras de mi DNI son 175,crear una variable que tenga ese numero

```
DNI=175
```

Consigna: Crear un vector que tenga todos los numeros enteros desde el 1 al 175

```
Secuencia_dni<-(1:175)
Secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175
```

##Ejercicio 2 de algoritmo Calcular la suma de todos los valores del vector Secuencia_dni

```
Total<-0
Valor_final<-length(Secuencia_dni)
for(i in 1:175) {
```

```
Total<-Total+i
Total
}
Total
```

```
## [1] 15400
```

##Ejercicio 3 Repetir el ejercicio anterior pero en Python

```
totalp = 0
for i in range(1,176):
   totalp += i
   print(totalp)
```

```
## 1
## 3
## 6
## 10
## 15
## 21
## 28
## 36
## 45
## 55
## 66
## 78
## 91
## 105
## 120
## 136
## 153
## 171
## 190
## 210
## 231
## 253
## 276
## 300
## 325
## 351
## 378
## 406
## 435
## 465
## 496
## 528
## 561
## 595
## 630
## 666
## 703
## 741
## 780
## 820
## 861
```

```
## 903
## 946
## 990
## 1035
## 1081
## 1128
## 1176
## 1225
## 1275
## 1326
## 1378
## 1431
## 1485
## 1540
## 1596
## 1653
## 1711
## 1770
## 1830
## 1891
## 1953
## 2016
## 2080
## 2145
## 2211
## 2278
## 2346
## 2415
## 2485
## 2556
## 2628
## 2701
## 2775
## 2850
## 2926
## 3003
## 3081
## 3160
## 3240
## 3321
## 3403
## 3486
## 3570
## 3655
## 3741
## 3828
## 3916
## 4005
## 4095
## 4186
## 4278
## 4371
## 4465
## 4560
```

```
## 4656
## 4753
## 4851
## 4950
## 5050
## 5151
## 5253
## 5356
## 5460
## 5565
## 5671
## 5778
## 5886
## 5995
## 6105
## 6216
## 6328
## 6441
## 6555
## 6670
## 6786
## 6903
## 7021
## 7140
## 7260
## 7381
## 7503
## 7626
## 7750
## 7875
## 8001
## 8128
## 8256
## 8385
## 8515
## 8646
## 8778
## 8911
## 9045
## 9180
## 9316
## 9453
## 9591
## 9730
## 9870
## 10011
## 10153
## 10296
## 10440
## 10585
## 10731
## 10878
## 11026
## 11175
```

```
## 11325
## 11476
## 11628
## 11781
## 11935
## 12090
## 12246
## 12403
## 12561
## 12720
## 12880
## 13041
## 13203
## 13366
## 13530
## 13695
## 13861
## 14028
## 14196
## 14365
## 14535
## 14706
## 14878
## 15051
## 15225
## 15400
```

##Ejercicio 4 ¿Cuanto tarda en correr el codigo del ejercicio 2?

```r
inicio<-Sys.time()
Total<-0
Valor_final<-10000000*length(Secuencia_dni)

for (i in 1:Valor_final){
    Total<-Total+i
}
Total
```

```
## [1] 1.53125e+18
```

```r
final<-Sys.time()
final-inicio
```

```
## Time difference of 28.23289 secs
```

Ejercicio 5

Repetir el ejercicio anterior utilizando la cabeza de Gauss

```r
system.time({
  n<-length(Secuencia_dni)

  #Aplicamos la formula de Gauss
  total_gauss <- (n*(n+1))/2
  print(total_gauss)
})
```

```
## [1] 15400
```

```
##    user  system elapsed
##   0.000   0.000   0.001
```

Ejercicios del profe:

1.4 Generar un vector secuencia

```r
# ---1. Generar secuencia con bucle ´for´---
#Primero inicializamos el vector
system.time({
  A<-0
  for (i in 1:50000) {
    A[i]<-(i*2)
  }
})
```

```
##    user  system elapsed
##   0.012   0.000   0.012
```

```r
# Verificamos los resultados
head(A)
```

```
## [1]  2  4  6  8 10 12
```

```r
tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

```r
system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})
```

```
##    user  system elapsed
##       0       0       0
```

```r
# Verificamos los resultados
head(B)
```

```
## [1]  2  4  6  8 10 12
```

```r
tail(B)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

1.5 Implementación de una serie Fibonachi o Fibonacci

```r
# Definimos la cantidad de elementos de la serie que queremos calcular
n <- 30

# --- Método 1: Bucle 'for' con pre-asignación
system.time({
  fib <- numeric(n)
  fib[1] <- 0
  fib[2] <- 1
```

```r
  for (i in 3:n) {
    fib[i] <- fib[i-1] + fib[i-2]
  }
})
```

```
##    user  system elapsed
##   0.004   0.000   0.003
```

```r
# Ver los primeros resultados
print("Primeros números de la serie:")
```

```
## [1] "Primeros números de la serie:"
```

```r
head(fib, 10)
```

```
##  [1]  0  1  1  2  3  5  8 13 21 34
```

```r
# --- Método 2: Función Recursiva

fib_recursiva <- function(k) {
  if (k == 0) return(0)
  if (k == 1) return(1)
  return(fib_recursiva(k - 1) + fib_recursiva(k - 2))
}

print("Tiempo ejecución recursiva:")
```

```
## [1] "Tiempo ejecución recursiva:"
```

```r
system.time({
  # Calculamos el elemento n de forma individual
  resultado_rec <- fib_recursiva(n)
})
```

```
##    user  system elapsed
##   0.792   0.000   0.800
```

1.6 Definición matemática recurrente

```r
# --- Resolver consigna con bucle 'while'

# Inicializamos la serie según la definición matemática
fib <- c(0, 1)
i <- 2 # Empezamos el conteo de iteraciones/posiciones

system.time({
  # El bucle sigue mientras el último número generado sea <= 1.000.000
  while (fib[i] <= 1000000) {
    i <- i + 1
    # Definición recurrente: f(n+1) = f(n) + f(n-1)
    fib[i] <- fib[i-1] + fib[i-2]
  }
})
```

```
##    user  system elapsed
##   0.003   0.000   0.003
```

```r
# --- Resultados ---
cat("El primer número mayor a 1.000.000 es:", fib[i], "\n")
```

```
## El primer número mayor a 1.000.000 es: 1346269
```

```r
cat("Se necesitaron", i - 1, "iteraciones (pasos) para superarlo.\n")
```

```
## Se necesitaron 31 iteraciones (pasos) para superarlo.
```

```r
# Mostramos los últimos elementos para verificar
tail(fib)
```

```
## [1]  121393  196418  317811  514229  832040 1346269
```

1.7 Ordenacion de un vector por metodo burbuja

```r
# --- 1. Creamos un vector desordenado ---
# Usamos un vector de 1000 números aleatorios
set.seed(123) # Para que el resultado sea siempre el mismo
vector_desordenado <- sample(1:1000, 1000)

# --- 2. Implementación del Método Burbuja ---
bubble_sort <- function(x) {
  n <- length(x)
  # El bucle externo recorre todo el vector
  for (i in 1:(n - 1)) {
    # El bucle interno compara pares adyacentes
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        # Intercambio de posición (el "burbujeo")
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
}

# --- 3. Medimos Performance del Burbuja vs R Nativo ---

cat("Tiempo con Método Burbuja:\n")
```

```
## Tiempo con Método Burbuja:
```

```r
system.time({
  vector_ordenado_burbuja <- bubble_sort(vector_desordenado)
})
```

```
##    user  system elapsed
##   0.062   0.000   0.062
```

```r
cat("\nTiempo con función sort() nativa de R:\n")
```

```
##
## Tiempo con función sort() nativa de R:
```

```r
system.time({
  vector_ordenado_R <- sort(vector_desordenado)
})
```

```
##    user  system elapsed
##       0       0       0
```

1.8 La penitencia de Newton

```r
# Definimos el límite de la suma (1*10^7 para que no tarde demasiado en R)
# Nota: 1*10^10 es un número extremadamente grande para un bucle for en R
n <- 1e7

# --- MÉTODO 1: La "Penitencia" (Bucle For / Fuerza Bruta) ---
# Este método suma uno por uno, como quería el profesor.
system.time({
  suma_iterativa <- 0
  for (i in 1:n) {
    suma_iterativa <- suma_iterativa + i
  }
})
```

```
##    user  system elapsed
##   0.162   0.004   0.168
```

```r
cat("Resultado Suma Iterativa:", suma_iterativa, "\n")
```

```
## Resultado Suma Iterativa: 5e+13
```

```r
# --- MÉTODO 2: La "Estrategia de Newton/Gauss" (Fórmula Analítica) ---
# Usamos la suma de una progresión aritmética: (n * (n + 1)) / 2
system.time({
  suma_analitica <- (n * (n + 1)) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

```r
cat("Resultado Suma Analítica:", suma_analitica, "\n")
```

```
## Resultado Suma Analítica: 5e+13
```